Theory of Computation

What can be done by computing machines?
Computability

What can realistically be done by computing machines?
Complexity

Where on earth to start?
Turing Machines

Exercise: Design a TM to increment unary number

Exercise

Design a TM to double a unary number

E.g. input ◊ 1 1 1 ◊ ◊ ◊ ◊ …
output ◊ 1 1 1 1 1 ◊ ◊ …

Be careful not to end up in an infinite loop !

We Can Prove That ...

- Any conventional programming language function can be implemented in a Turing Machine
- There are functions which are not Turing Computable
  - So there are programming tasks which cannot be written



Building Blocks for our Proofs

- Turing machines can be merged

- We can makeTuring Machines that do basic tasks:
  - finding a letter in a string
  - shifting a string to the right
  - etc

Combining Machines

- Simple General algorithm for combining any two machines, M1 and M2:
  - Change the halt state of M1 not to halt.
  - For each letter, *x*, in M1's alphabet, create the transition *x/x* to M2.

"Complex" Machine Combiner

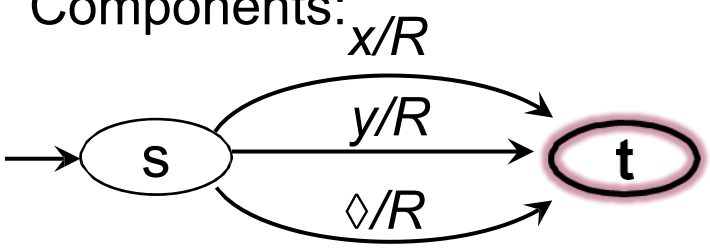
- Given machines, M_1, M_2 ... M_n
  - Remove the initial state designation from M_2 ... M_n.
  - Remove the halt designation from M_1 ... M_{n-1}.
  - For each old halt state, S_h, and each *x* in the machines' alphabet:
    - If the new machine should halt upon reaching S_h, with current symbol, *x*, draw an arc labelled *x/x* from S_h to the M_n halt state.
    - If upon reaching S_h, with current symbol, *x*, the new machine should transfer control to M_i, draw an arc *x/z* from S_h to S_q of M_i where $\delta(S_{intial}, x) \rightarrow (S_q, z)$.

Advantages of Complex Combiner

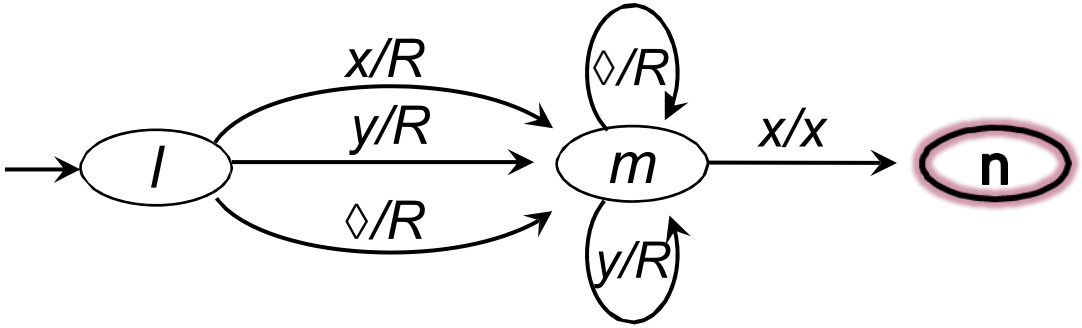
- Flexibility:
  - can combine multiple machines in any order
  - can halt conditionally upon encountering a particular symbol in an old halt state
- Efficiency
  - Except for the starting machine, we by-pass all the initial states of the machines.

Example of Combining Machines

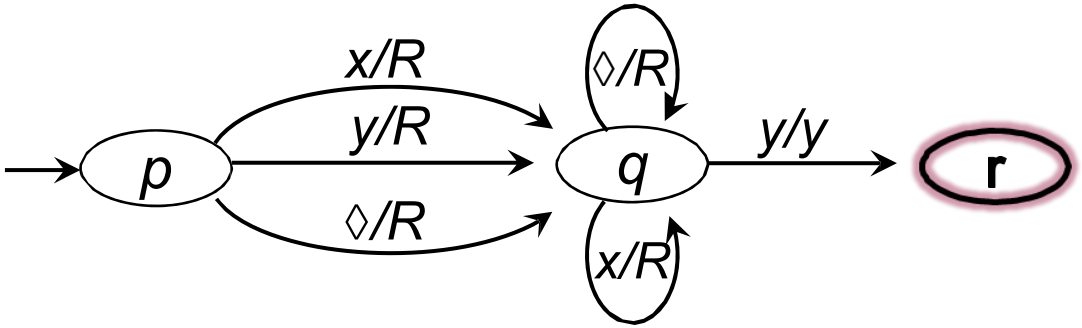
- Find the second occurrence of the non-blank symbol to the right of the initial head position. The tape alphabet is {x, y, ◊ }.
- Components:



Moves the tape head one cell to the right



Finds the first x to the right of the current cell



Finds the first y to the right of the current cell

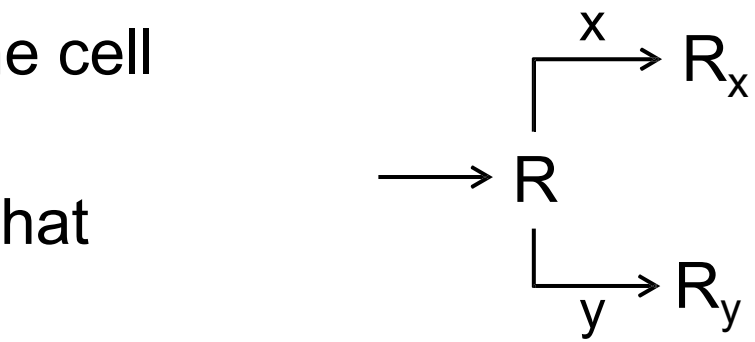
Solution Summary

Find the second occurrence of the non-blank symbol to the right of the initial head position. The tape alphabet is {x, y, ◊ }.

Node R is a machine that moves its tape head one cell to the right

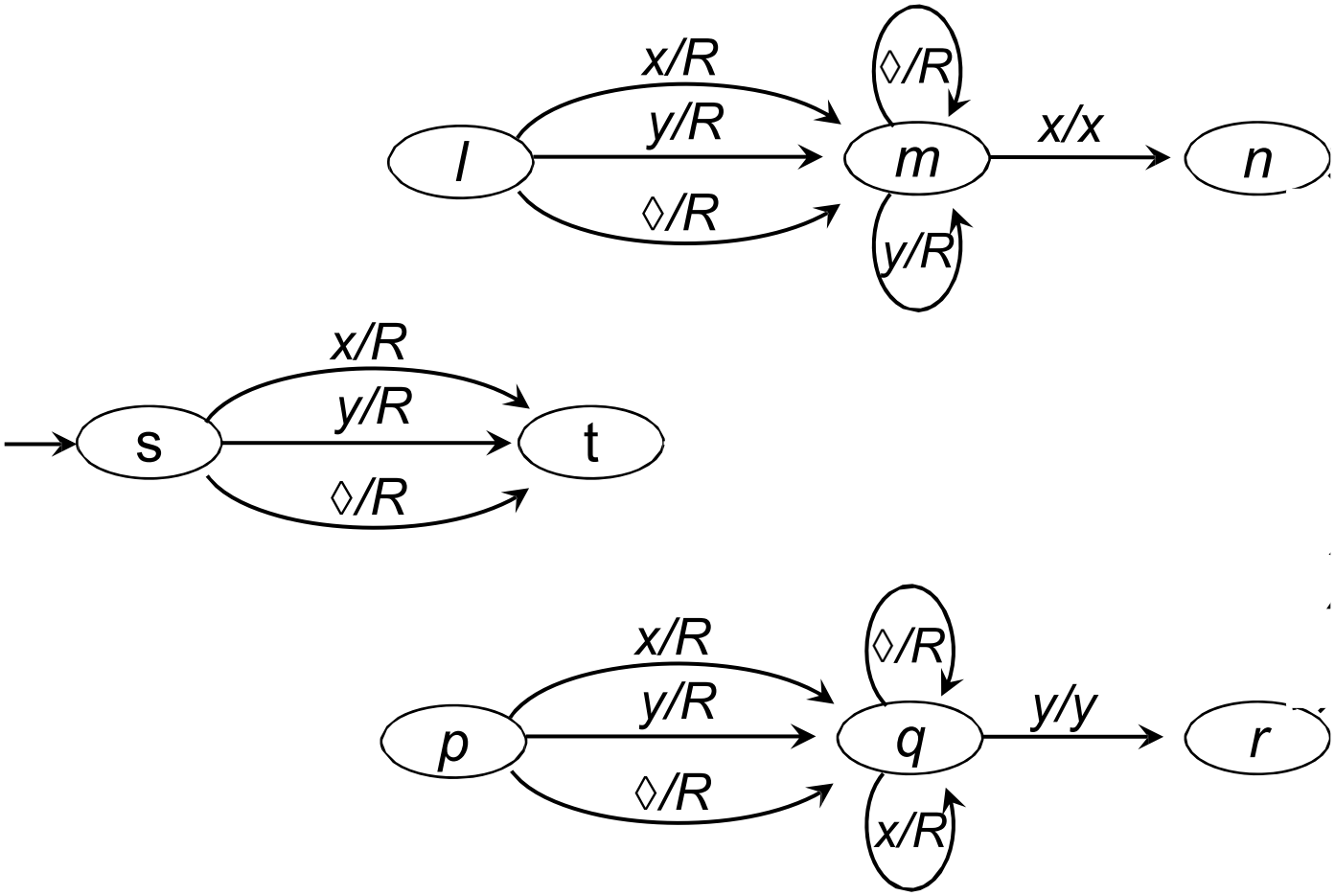
Node R_X is a machine that searches for another x

Node R_Y is a machine that searches for another y



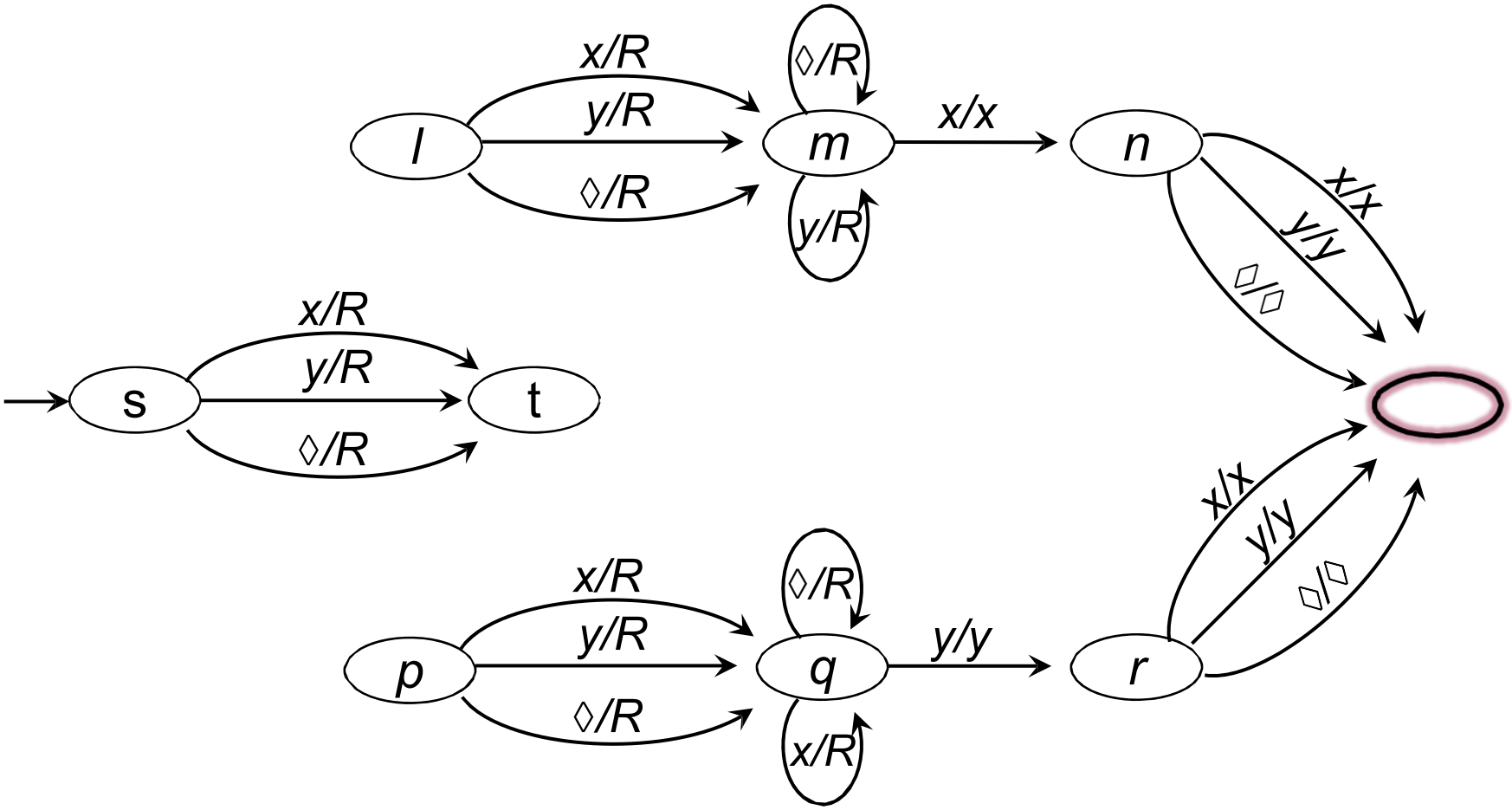
Solution

Find the second occurrence of the non-blank symbol to the right of the initial head position. The tape alphabet is {x, y, ◊ }.



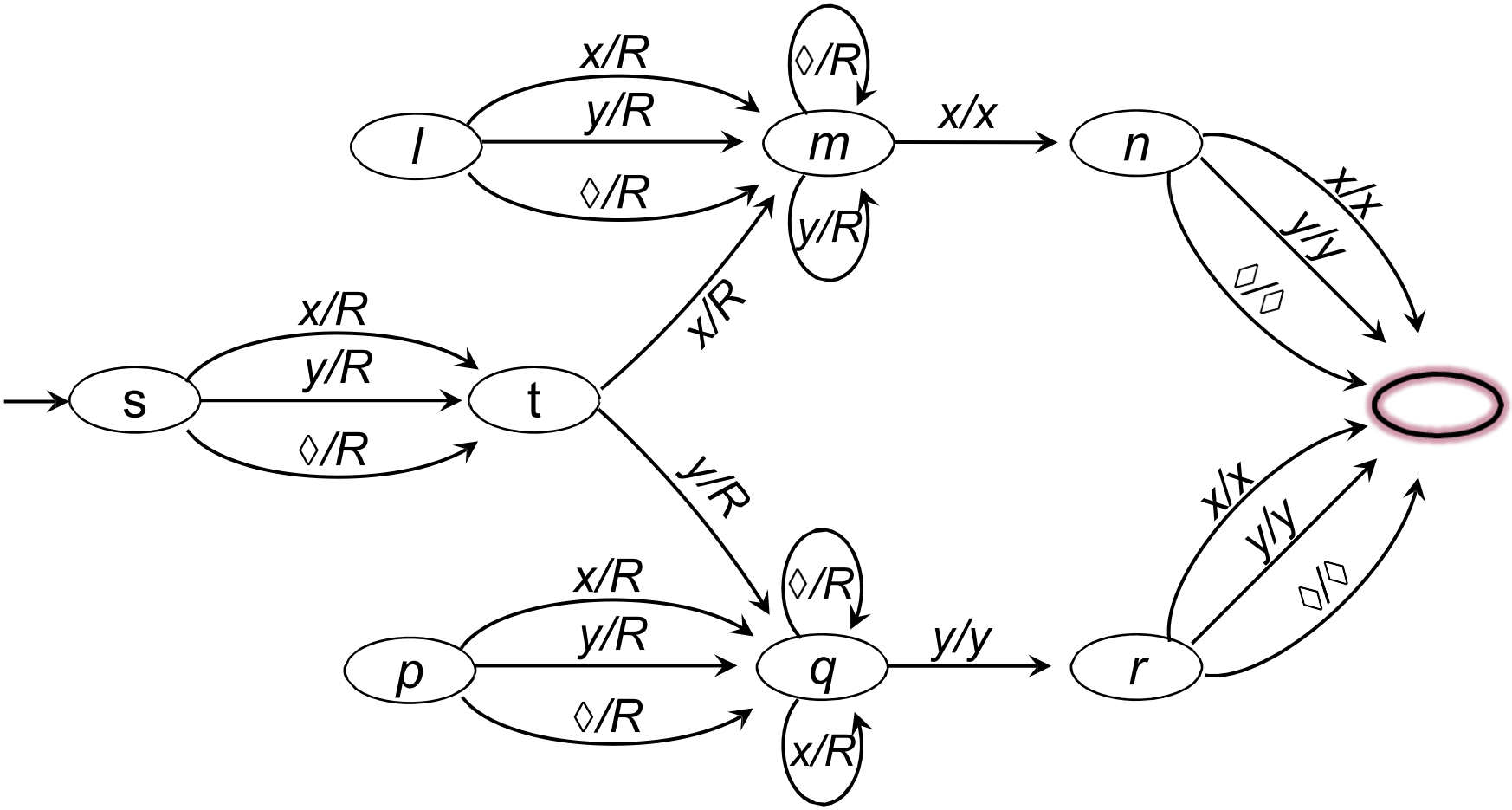
Solution

Find the second occurrence of the non-blank symbol to the right of the initial head position. The tape alphabet is {x, y, ◊ }.



Solution

Find the second occurrence of the non-blank symbol to the right of the initial head position. The tape alphabet is {x, y, ◊ }.



Basic Turing Machines

R_x: Move right until x is found

L_x: Move left until x is found

$L_{\sim X}$: Move left until (not x) is found

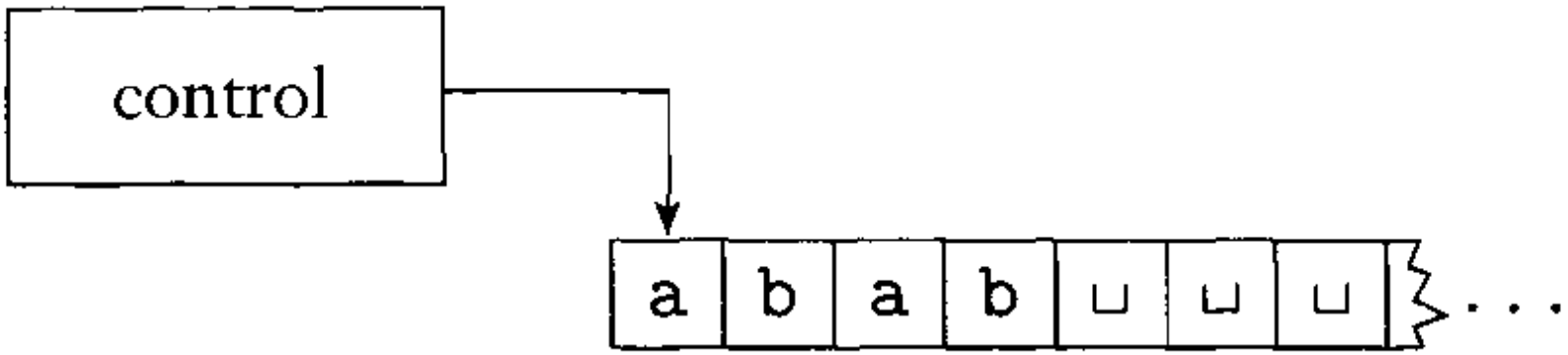
$R_{\sim x}$: Move right until (not x) is found

S_R: Shift the entire string right

- E.g. ◊yxy◊◊xxy◊◊ becomes
  ◊◊yxy◊xxy◊◊

S_L: Shift the entire string left

Schematic of a Turing Machine

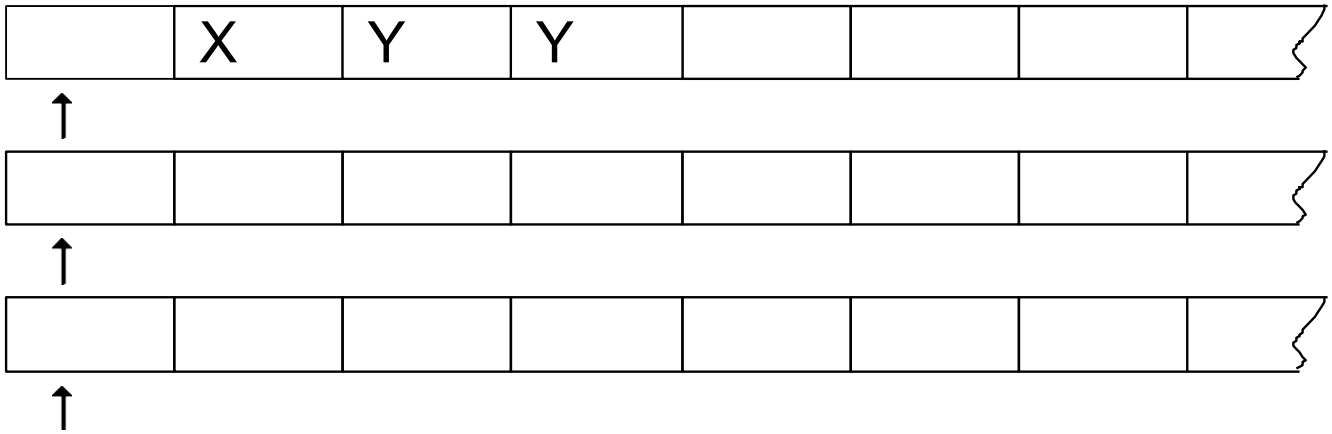


"Make everything as simple as possible, but not simpler"

Question: Is the Turing Machine too simple? Or will extending it a little give it more power?

Multi-Tape Turing Machines

State	Tape 1	Tape 2	Tape 3	Action
S1	X	X	X	S2, 2L
S1	X	X	Y	S2, 1R
S2	Y	X	Y	S1, 2Y

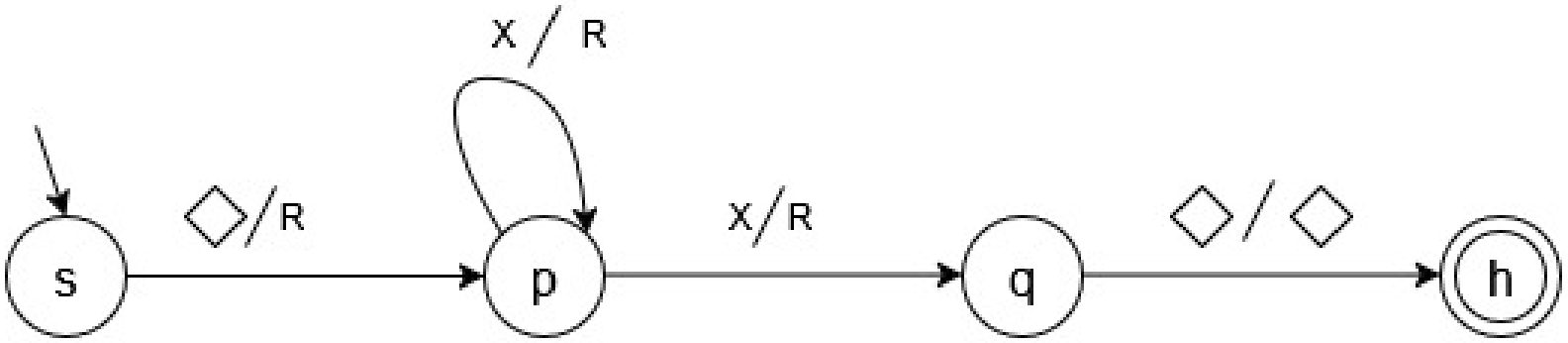


Multi = single: Summary of Proof

1. Represent tapes as an array
2. For each possible column in the array, define a symbol.
3. Setup the tape so that there's a precise correspondence between the newly defined symbols and the tapes of the multi-tape machine.
4. Create the compound states.
5. Execute the transition algorithm

Multi-tape Turing Machines are NOT more powerful/capable than single-tape machines!

Nondeterministic Turing Machines



A Nondeterministic Turing Machine to accept X^+

It is nondeterministic because there are 2 arcs from state p both labelled with X

State table includes both transitions below !!

Current State	Symbol	Action	Next State
p	X	R	p
p	X	R	q

Nondeterminism = Determinism

A NTM can be simulated on a deterministic single tape TM

DTM can simulate each transition of the NTM, making multiple copies of the simulated state when multiple transitions are possible, and simulating them all in parallel

Nondeterministic Turing Machines are NOT more powerful/capable than deterministic machines!